



SDEV 2451

Full Stack Development

Introduction to Full Stack Development - 2

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments
- No Cheating
- Be a good classmate
- Don't waste your time
- Show up to class

Agenda

On the right is what we will cover today.

- Configure Settings and Project URLs
- Create Models and Load Fixtures
- Create Serializers
- APIView and ModelViewSet
- Wire Up the Router
- Test with Postman
- Challenge — Filtering and Custom Actions
- Key Takeaways

Configure Settings and Project URLs

Register DRF in Django and point the project URLs at the app before writing any view code.

```
# settings.py
INSTALLED_APPS = [
    # ... django built-ins ...
    "rest_framework",
    "blog",          # your custom app
]

REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.AllowAny",
    ],
}
```

```
# project/urls.py
urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/v1/", include("blog.urls")),
]
```

Configure Settings and Project URLs (continued)

- `"rest_framework"` activates DRF's browsable API, view classes, and settings
- `DEFAULT_PERMISSION_CLASSES: AllowAny` makes every endpoint public — fine for a review example, tighten before deploying
- `include("blog.urls")` delegates all `/api/v1/` traffic to the app's URL file, which we create in Step 5

Create Models and Load Fixtures

Define the domain models that the API will expose, then seed the database with fixture data for testing.

```
# blog/models.py
class Author(models.Model):
    name = models.CharField(max_length=150)
    email = models.EmailField(unique=True)

class Article(models.Model):
    author = models.ForeignKey(Author, on_delete=models.CASCADE,
                              related_name="articles")
    title = models.CharField(max_length=200)
    published_at = models.DateTimeField()
    word_count = models.PositiveIntegerField(null=True, blank=True)
```

```
python manage.py makemigrations
python manage.py migrate
python manage.py loaddata authors articles
```

Create Models and Load Fixtures (continued)

- `unique=True` on `email` enforces uniqueness at the database level — DRF returns a clean `400` if a duplicate is sent
- `on_delete=CASCADE` deletes related articles when the author is deleted; use `PROTECT` to prevent deletion instead
- `related_name="articles"` allows `author.articles.all()` to traverse the reverse relationship
- `null=True`, `blank=True` on `word_count` lets articles be created before they're finished — update later
- `loaddata` must be run in dependency order (authors before articles); running it twice on the same DB fails with a primary-key conflict

Create Serializers

Serializers convert model instances to JSON for responses and validate incoming JSON before saving.

```
# blog/serializers.py
class AuthorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Author
        fields = "__all__"

class ArticleSerializer(serializers.ModelSerializer):
    author_detail = AuthorSerializer(source="author", read_only=True)

    class Meta:
        model = Article
        fields = ["id", "author", "author_detail",
                  "title", "published_at", "word_count"]
        extra_kwargs = {
            "author": {"write_only": True},
        }
```


Create Serializers (continued)

- `fields = "__all__"` exposes every model field — fine when there's no sensitive data
- `author_detail = AuthorSerializer(source="author", read_only=True)` nests the full author object inside the article response
- `"author": {"write_only": True}` hides the raw FK integer from reads while still accepting it on writes — **integer in, object out**
- The `source="author"` argument tells DRF which model attribute to use as the data source for the nested serializer

APIView and ModelViewSet

DRF provides two view patterns: `APIView` for hand-written logic, and `ModelViewSet` for standard CRUD with minimal code.

```
# blog/views.py

class BlogStatsView(APIView):
    def get(self, request):
        return Response({
            "total_authors": Author.objects.count(),
            "total_articles": Article.objects.count(),
        })

class ArticleViewSet(ModelViewSet):
    serializer_class = ArticleSerializer
    filter_backends = [SearchFilter]
    search_fields = ["title", "author__name"]

    def get_queryset(self):
        return Article.objects.select_related("author").all()
```

APIView and ModelViewSet (continued)

- `APIView` maps HTTP methods directly to Python methods (`get` , `post` , etc.) — best for custom endpoints that don't map to a single model
- `ModelViewSet` generates `list` , `create` , `retrieve` , `update` , `partial_update` , and `destroy` from just `serializer_class` and a queryset
- `SearchFilter` adds a `?search=` query parameter; DRF performs a case-insensitive `icontains` across all `search_fields`
- `select_related("author")` fetches the related author in the same SQL JOIN — prevents the **N+1 problem** when the serializer reads `author_detail` for every article

Wire Up the Router

`DefaultRouter` turns ViewSets into a full URL set with a single `register` call per resource.

```
# blog/urls.py
from rest_framework.routers import DefaultRouter
from blog.views import ArticleViewSet, AuthorViewSet, BlogStatsView

router = DefaultRouter()
router.register("articles", ArticleViewSet, basename="article")
router.register("authors", AuthorViewSet, basename="author")

urlpatterns = [
    path("stats/", BlogStatsView.as_view(), name="blog-stats"),
    path("", include(router.urls)),
]
```

Wire up the Router (continued)

- `DefaultRouter` generates two patterns per `ViewSet`: a list endpoint (`/api/v1/articles/`) and a detail endpoint (`/api/v1/articles/{id}/`), plus a browsable API root at `/api/v1/`
- `basename` is required when using `get_queryset()` instead of a static `queryset` attribute — the router needs it to name the generated URL patterns (`article-list` , `article-detail`)
- The manual `stats/` path is registered before `include(router.urls)` because it is a standalone `APIView` , not a `ViewSet` action

Test with Postman

Run the server and use Postman to exercise every endpoint.

```
python manage.py runserver
```

Action	Method	URL
List articles	GET	/api/v1/articles/
Create article	POST	/api/v1/articles/
Retrieve one	GET	/api/v1/articles/1/
Update article	PATCH	/api/v1/articles/1/
Delete article	DELETE	/api/v1/articles/1/
Search	GET	/api/v1/articles/?search=django



ESSENTIAL
TO ALBERTA



Challenge — Filtering and Custom Actions

1. Add trip filtering by vehicle and driver

- Add `django-filter` and configure `DEFAULT_FILTER_BACKENDS` in `REST_FRAMEWORK` settings
- Create a `FilterSet` that filters by foreign-key fields and a date range using `__gte` / `__lte` lookups
- Apply `filterset_class` to the `ViewSet` and test with `?vehicle=1&start_time__gte=2026-06-01T00:00:00Z`

2. Add a "publish" endpoint to `ArticleViewSet`

- Add a custom `@action(detail=True, methods=["patch"])` called `publish`
- The action should set `published_at` to the current time if it is currently `null`
- Return a `400` with `{"detail": "Already published"}` if `published_at` is already set
- Test at `PATCH /api/v1/articles/{id}/publish/`



ESSENTIAL
TO ALBERTA



Key Takeaways

- `APIView` — define `get()`, `post()`, etc. as methods; best for custom endpoints that don't map to one model
- `ModelViewSet` — generates all six standard CRUD actions from just `queryset` and `serializer_class`
- `DefaultRouter` — auto-generates list and detail URL patterns per `ViewSet`; `basename` required with `get_queryset()`
- `SearchFilter` — adds `?search=` with case-insensitive `icontains` across `search_fields`; no extra packages needed
- **Nested serializers** — use `NestedSerializer(source="field", read_only=True)` to embed related objects; pair with `write_only=True` on the FK for a clean integer-in, object-out interface
- `select_related` in `get_queryset()` — prevents the N+1 problem when a serializer traverses a `ForeignKey` or `OneToOneField`

WE
ARE ESSENTIAL
TO ALBERTA





Example

Let's go do an example together

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS